

What is worth doing next

ECO5040W Group 3 • code and specification

For the group

27 September 2026

Status, 27 September 2026. The memo lookup, decrypting a wallet secret only when a transaction is about to be signed, the two specification corrections, and a saved enqueue log are now in the repository. Two worker runs that overlap can still both pay. What follows is why those items were the ones worth doing. It is not a new task list. The live demo, the slides, and formatting cleanup are still outside this note.

This note is a reading of the current code and of Kerry's specification against the course brief. It is not a grade.

The figures below are conservative estimates of the brief's weights. They describe this repository, not a marker's final number.

Slice	Weight	Estimate	What is holding it
Specification	20%	14/20	Content matches the code. Two sentences are still loose.
Web application	35%	30/35	The journey is implemented.
XRPL and queue	20%	14/20	A crash can submit a second payment.
Security	10%	6/10	Keys decrypt on every wallet load.
Performance	10%	7.5/10	HTTP results are saved. Ledger timings are not.

Taken together, that is about 72 of the 85 marks in these five slices. The remaining presentation mark is outside this note.

1 Already in good shape

The remittance path is built and the specification now describes it without the earlier overclaims. A marker can follow register, login, KYC, beneficiary, quote, cash-in, Redis settlement, wallet, and cash-out in both the code and the document.

Fees, limits, and the worked R1,000 example match the code: a R25 fixed fee, 1% of the amount, a 2% exchange-rate margin, and an estimated 1.5% cash-out fee. The prototype does not claim to be cheaper than WorldRemit, under the 3% target, licensed, or an anti-money-laundering system. It does not claim to enforce the Reserve Bank's R2 million annual allowance. UCTUSD is named as the token that is actually paid, with the issuer left in configuration, which is the fallback the course notes allow.

Passwords are bcrypt hashes. Ledger secrets and the KYC identification number are encrypted with separate keys that are not stored in the database. Wallet responses do not include secrets.

The September HTTP load test is backed by saved Locust output: 2,218 requests, no failures, and a slowest request under two seconds.

None of that needs to be rewritten.

2 What still costs marks

Three gaps are real. Two smaller specification sentences are worth a short edit. Everything else in the critic list is either already honest or not worth the risk.

2.1 A second ledger payment is still possible

The brief says a duplicate settlement message must not credit the recipient twice. The code has one settlement row per remittance, and it will not pay again once the transaction hash is stored. That is not the whole requirement.

The worker submits the payment, waits until the ledger accepts it, and only then saves the hash and credits the wallet in one database commit. If the process dies after the ledger has accepted the payment and before that commit, the hash is still empty. The next run does not see a hash, so it can submit a second payment. The automated tests do not catch this, because the fake ledger returns the same hash for a given remittance and never pays twice.

The specification already says this. Saying it again will not recover the marks. The payment lookup below would.

2.2 Private keys are decrypted too early

Secrets are encrypted at rest, and they are not sent back by the API. The brief also says a key may be decrypted only by the code that signs the ledger transaction. That part is not true yet. Loading a wallet row decrypts the secret immediately, and the settlement and cash-out code then pass the plaintext into the payment function. The specification already admits this, which is the right thing to have written. Removing that sentence before the code changes would make the document false.

This is a smaller mark than the double payment. A careful change might move security from about 6/10 to about 7/10. A careless one can break settlement and cash-out while the tests still pass, because the tests do not check that a seed is real.

2.3 Two specification sentences

The history screen does not show the quote breakdown. The sender's list at `/app/history` is the amount, the status, the tracking reference, and the date. The breakdown, the timeline, the hash, and the QR code are on the track page and on the quote page. The FR-28a cell currently reads as if the history page shows all of that. A marker who opens History will mark the cell wrong.

The wallet section explains the mechanics, but it never says in one sentence that the team used both options the brief allowed: a separate Testnet account for each linked recipient, and an internal

ledger for the spendable balance. The brief asked for that choice to be stated.

2.4 The performance file is only half evidenced

API response times, requests per second, the charts, and the login bottleneck are in saved Locust files from 7 September. Those should be left as they are. The queue rates and the Testnet timings (the 5-payment and 15-payment runs) appear only as sentences in `PERFORMANCE_TESTING.md`. There is no saved program output next to them. They should stay labelled as timing notes, not as a log the marker can open.

3 What to do, in this order

1. **Close the double payment in code.** Before a payment is submitted, ask the Testnet for transactions on the platform account and look for one that already carries this remittance's id in the memo. The memo is already written on the way out. Nothing reads it on the way back. If that payment is found and it succeeded, use its hash and do not submit again. Then run the existing commit that stores the hash and credits the wallet. Do not use the helper that marks a job complete when a hash already exists: that helper skips the credit, and in this crash the credit has not happened yet.

The same lookup belongs on the payment function used for cash-out, because a cash-out burn uses the same memo pattern with the cash-out id.

This does not make two overlapping worker runs safe. It does make a retry after a crash adopt the payment that already happened. Describe it that way. Do not call it exactly-once until the lookup is in the code.

A test can fake the ledger history: a second attempt must not submit, must return the existing hash, and must credit the wallet once. One real Testnet payment is still needed afterwards: settle, clear the stored hash, run the worker again, and check that the explorer shows a single payment.

Conservative effect: about 2 to 4 of the 20 XRPL marks. The rest of that slice is already demonstrated.

2. **Correct the two specification sentences, and nothing else around them.** Replace the FR-28a implementation cell with: the sender list shows amount, status, and tracking reference; the quote breakdown, timeline, hash, and QR code are on the track page and the quote page, not on the history page.

Add one sentence to the architecture paragraph: the prototype uses both wallet options in the brief, a separate Testnet account per linked recipient and an internal spendable ledger, and the on-chain figure is not a live trust-line balance.

Do not rewrite the business-problem essay, the competitor percentages, or the diagrams.

3. **Save one queue measurement. Do not rerun the HTTP test.** From backend, run:

```
python -m scripts.benchmark_settlement -enqueue-only
```

Save the output as a text file under `backend/perf/results/` and commit it. That command enqueues 200 messages and does not touch Testnet. In the performance note, say the file is the enqueue measurement and that the rate depends on the machine.

If someone still has the terminal log from the 7 September or 24 September settlement runs, commit that log too. If nobody has it, do not type the old averages into a new file and call it a log. Leave those averages in the markdown as samples.

4. **Narrow key decryption only if the money path can be retested in the same sitting.**

Keep the KYC identification number as it is: encrypted in the database, shown to the account owner and to an administrator.

For the two secret columns only, load ciphertext and decrypt inside the signing call: settlement, cash-out completion, the treasury TrustSet, and the retry path that rebuilds a recipient wallet from the stored seed. Do not change decryption for every encrypted column, or KYC responses will start returning ciphertext.

Do not decrypt inside the low-level submit function if tests pass it a plaintext fake seed. Decrypt at the four call sites above, and pass the plaintext in.

If that cannot be tested against settlement, cash-out, and the wallet-retry test before it is committed, leave the code and keep the honest sentence in the specification. The mark movement is small. A broken payment path costs more.

4 What not to change

- Do not turn the worker into a process that stays running. The admin action that drains the queue already meets the brief.
- Do not switch the token from UCTUSD to RLUSD. The issuer is already configuration, and the course notes allow the course token.
- Do not add sanctions screening, the annual R2 million cap, or a third stored role.
- Do not rerun the 50-user Locust test. The saved CSV already matches the write-up.
- Do not treat 15 sequential Testnet payments as a reliability rate, and do not add a second worker to manufacture a concurrency number.
- Do not build a separate key store, and do not encrypt name, address, or contact fields.
- Do not delete the specification's sentence about decrypt-on-load until the code in item 4 is actually in place.
- Do not redraw diagrams or replace the competitor table. Those figures were checked and should stay.

5 If there is only time for one thing

Do item 1, the ledger lookup before a second payment, and item 2, the two sentences. Item 3 is a short command if Redis is up. Item 4 waits until someone can watch a settlement and a cash-out succeed after the change.